

Concurrent, Overlapping Development and the Dynamic System Analysis of a Software Project

Jaideep Ghosh

Abstract—In recent years concurrent, overlapping development activities of engineering tasks have proved to be a cost-effective and time-efficient way to build products geared toward meeting the demands of a competitive and quality-conscious market. In this study, we examine the structure and dynamics of a software project in information technology industrial research and development that has this methodology as its foundation. We first describe a number of software engineering aspects of the project that elucidate how various development activities in its concurrently unfolding component projects are overlapped and integrated to gradually build the product over time. Using the mathematical tools of dynamic systems analysis, we next derive a number of results that characterize the long-term, stable behavior of the project. These results are then used in a numerical simulation to illustrate a procedure by which relevant resources can be distributed across the components in a way that maintains continued stability of the project over time. Our modeling approach is flexible enough to allow the theoretical results and the simulation techniques to be easily generalized to study, in real time, the stability, maintenance, and management of similar concurrent projects. The usefulness of the method lies in motivating and refining managerial decisions that drive overlapping development activities of such projects.

Index Terms—Activity overlapping, asymptotic stability analysis, concurrent development, dynamic systems modeling, numerical simulation, parallel development, resource distribution, simultaneous engineering.

I. INTRODUCTION

A NUMBER of major projects today in information technology (IT) software research and development (R&D) are set up in the architecture of concurrent, overlapping development (COD). The more economical and efficient use of resources in this style of development and management helps organizations build products faster [1], [2]. Basically, the COD practice utilizes the benefits of simultaneous development of a number of well-selected components together with the continually progressing overlaps of their development activities through frequent exchanges of information [3]. The components can be any relevant engineering entities, partial or complete. Consider the example of the manufacturing process of an airplane's door. The different components that may be developed concurrently in an overlapped fashion could include the electronic door-lock system, the door handle, the door body, the computerized interface of the inside door panel, and so on. Even the

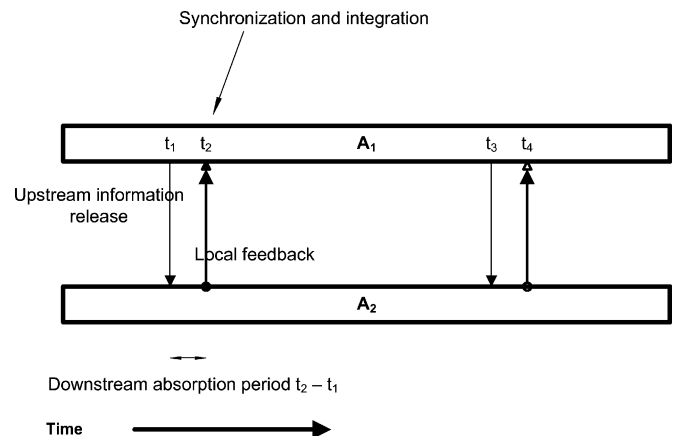


Fig. 1. Temporal evolution instances of two concurrent, overlapping activities.

lock component might consist of further subcomponents, such as modules for checking air pressure inside and outside the airplane, mechanisms for communication with a central computer housed in the cockpit, interfaces with ground-based control systems, emergency operating features, etc. The COD activities in the components entail several important engineering and management decisions. For example, what coevolving activities in the components can be allowed to exchange information among themselves; what information can be frozen early for release to downstream development; how different activities in locally interacting components can be coordinated; how the upstream and downstream development work can be reconciled, and subsequently integrated, etc. These are all critical issues that the management in charge of COD must successfully address in order to evolve the project stably over time.

The different intercomponent overlapping activities proceed by releasing information upstream (earlier time), and subsequently absorbing, synchronizing, and integrating the information downstream (later time) through local feedback processes. Fig. 1 depicts a simplified scenario in which two coupled activities A_1 (for example, building a conceptual design) and A_2 (for example, developing a small prototype application) are concurrently executed in an overlapped fashion through a transaction initiated with the release of partial upstream information by A_1 , which is then absorbed by A_2 for downstream development. By means of local (i.e., affecting this specific interaction at one small period of time) feedback, the development of A_1 and A_2 are then integrated, synchronized, and reconciled for future progress in time. The pattern repeats itself over cycles of overlapping development. The integration of the activities can be periodic or aperiodic, synchronous or asynchronous. Also, depending on the nature of the evolution and the sensitivity of the

Manuscript received April 16, 2008; revised September 18, 2008 and February 5, 2009. First published August 18, 2009; current version published April 21, 2010. Review of this manuscript was arranged by Department Editor J. Sarkis.

The author is with the Information Systems and Operations Management Department, School of Management, George Mason University, Fairfax, VA 22030 USA (e-mail: jghosh@gmu.edu).

Digital Object Identifier 10.1109/TEM.2009.2023138

TABLE I
DESCRIPTION OF SYMBOLS USED IN THIS PAPER

C_X	The name of an arbitrary business organization that undertakes the development of a COD project.
P_X	The COD project.
N	The total number of component projects selected for COD.
$C_i, i = 1, \dots, N$	A component project of P_X .
P	The final product developed in P_X .
$\mathcal{G}_X$	The project group of P_X that controls various overlapping development processes.

activities, the type of overlapping can be iterative, preemptive, divisive, or distributive [4].

A common application of COD in IT is the development of one major software product along with a suite of related, subsidiary products. Many companies follow this practice, because it is an economical way of sharing capital, labor, energy and any other resources that may be needed in the process of development. COD has found use in single as well as in multicomponent projects. In the latter case, a small group of component projects are combined in overlapping, integrative development to build a major software product gradually over time. The application of COD to large-scale, multicomponent software projects in the industry has often been found to give rise to smoother and more time-efficient development and management. With all the benefits of COD notwithstanding, it must be remembered that there are many tasks for which the practice may not be suitable. The practice depends very critically on the nature of the components selected for parallel development and the corresponding activities that are to be overlapped. The making of a number of judicious and timely decisions plays a vital role in the management. Failure to choose the proper components for simultaneous development or to identify their activities to be overlapped could spell disaster, leading to a gross increase, instead of the desirable reduction, in development costs or the time or both [4]–[7]. Careful consideration, cooperative planning and dynamic adjustment, as instruments of managerial practices, are necessary to minimize the risks associated with overlapping.

The evolution of a small but important class of these multicomponent projects in software R&D can be described by a model introduced in the present work, which we term generically the COD model. It is based on a common set of methodologies and software engineering processes derived from two industrial R&D projects undertaken by two independent, large business organizations in the U.S., where the present author was personally involved. To preserve confidentiality and the proprietary rights of the organizations, their names, the internal details of the concerned projects and the software products are not disclosed in this article. However, for the purpose of identifying the entities of interest in the present paper, we employ some generic symbols that are displayed in Table I.

The fundamental idea behind the COD project that we study in this paper can be stated as follows: at some time, say $t = 0$, C_X undertakes the development of a major project P_X based on a number, say N , of small to mid-sized R&D projects. These projects, called the components C_i , $i = 1, \dots, N$, of P_X are scheduled to evolve concurrently in time with their develop-

ment activities overlapping spatially through mutual dependencies. Just as any ordinary IT project, each C_i builds a software product in accordance with commercial demands involving specific business rules and requirements. It, therefore, has a fixed time span of development, after which its functional product is released onto the market. Now, simultaneous with the ongoing development of these C_i 's, various programs, interfaces, frameworks, and modules used in their different evolving applications are appropriately combined and engineered in an overlapped fashion in the project P_X to build a major product P , which, a result of an innovative COD endeavor, is marketed independently by C_X . The product itself is believed to be able to operate in a broad IT market, with the prospect of earning considerable revenue for C_X . When involved in this kind of development, the C_i 's are not allowed to function in isolation, but they must be reconciled together, coordinated and managed within the setting of a project group. This group, which is denoted by $\mathcal{G}_X$, is actually an abstract entity that symbolizes the functional domain of a set of engineering processes and rules appropriate to overlapping development. Ultimately, the COD structure gives rise to a network that binds the components together in $\mathcal{G}_X$ through channels of information exchange gradually opened up by the overlapping development activities. Within the purview of $\mathcal{G}_X$ lies the most crucial activities of COD engineering management. Some of these are the determination of the dependency measures of the C_i 's and the estimated amounts of output that each component should contribute incrementally in time to P_X ; the coordination plans and procedures of upstream and downstream exchanges of information during design and development; the anticipated extent of the resulting iterative work and rework in the concerned components; the framework created by the requirements of P_X that constrain the development of its components, etc. The relevant software engineering aspects are discussed later in the paper.

One of the principal difficulties in successfully managing a complex COD enterprise is to know how to distribute project resources in the various components participating in COD. When a COD project is undertaken by a single organization, it is very likely that there is a common pool of resources available to the project, although such resources may well be distributed over multiple spatial locations. Due to the overlapped nature of the interactions intrinsic to COD, it is not always possible to know ahead of time how certain resources should be distributed in the components, so that the process of information exchange between them becomes smoother and more streamlined, requiring fewer downstream iterations. For example, a component that has

a need for trained or skilled manpower may be allocated substitutional technological resources (automated applications, for instance) in larger proportions. Similarly, a subset of the components having large interdependencies or in need to frequent information exchanges may be allocated more flexible human resources, such that a few knowledgeable design architects may work across these components at the same time. Now, while the resources are being distributed in this manner, the ongoing time evolution of the project must remain stable. This is actually a dual problem requiring a solution that addresses both issues. This study uses the methods of dynamic systems analysis to study this problem in particular. The usefulness of the work lies in establishing a preliminary step for modeling the first-order behavior of this type. Based on the theoretical results, a numerical simulation of a simplified version of P_X illustrates, within a limited framework, how resource distribution can be simultaneously balanced with the maintenance of stability in the project. The tools introduced in this work can be applied to sufficiently refine concerned managerial procedures and decision-making for the long-term sustainability of these projects.

The balance of the paper is organized as follows. Section II briefly reviews earlier work and the related literature on the activities of overlapping product development in general industrial applications. In Section III, we outline the main software engineering principles of the COD model of P_X . This section is neither exhaustive nor sufficiently detailed to cover all aspects of P_X 's development but serves only as an introductory background for the relevant ideas used in the present work. A dynamic systems analysis of P_X is presented in Section IV with a number of useful, quantitative results to characterize the long-term, stable behavior of the project. These results are used in Section V in a simulation that illustrates the characteristics of the long-term stability and the balanced distribution of resources in the project. Section VI concludes the paper, and also indicates some limitations of this study that reveal directions of future research in this area.

II. RELATED PREVIOUS WORK

The practice of concurrent, overlapping execution of activities in any engineering enterprise has two major objectives: time shortening and cost reduction. While there are obvious relations between the two, in many cases, depending upon the nature of the task at hand, these two aspects can be treated independently. Early researchers were primarily concerned with the time issue, as many manufacturing firms in Japan, Europe, and the U.S. were gradually introducing concurrent engineering practices to shorten product development cycles. Not every project succeeded in this regard; however, in the ones that did perform well, the effect of substantial reduction in development costs as a by-product of the emerging technological endeavor was realized subsequently. The modern viewpoint is that, both time shortening and cost reduction are simultaneously achievable, if certain principles and guidelines are observed. Today, the entire practice of this engineering methodology with all attendant organizational activities are collectively termed *simultaneous engineering* [8], [9]. Although the COD model of P_X in the

particular setting of IT software R&D is a specialized example of simultaneous engineering, many aspects and features of the latter are not relevant to this study. In this section, we, therefore, briefly survey some specific results, guidelines, and recommendations found in related previous work that have a direct bearing on the development of P_X .

It is well known that many common product-development operations in the industry are sequential in nature [10], [11]. This signifies that the flow of information between interacting engineering activities is unidirectional. Thus, each step or phase in a process must be completed before the next one can be commenced. For example, the design of a small mesh of networks in a large telecommunications backbone can only begin *after* all the functional requirements of the peripheral routing system have been put into perspective. This is the normal engineering practice when the time between conception and implementation is long enough to allow sequential operations to take place. However, in many industrial applications, the sequential execution method proves to be both time-consuming and impractical. In order to speed up the development process, concurrently executing activities are appropriately overlapped. In this context Blackburn [12] explains that overlapping can shorten project cycles by stimulating parallel activities and also by avoiding much time-consuming rework through the release of information early in development so that engineers are able to "begin working on different phases of the problem while final designs are evolving." Although such a procedure cannot be employed in all stages of product development, the author concludes that, whenever and wherever feasible, the approach should work better and faster than its sequential counterpart.

Next, in applications of product innovation in R&D, Mansfield *et al.* [6] examined the effects of overlapping stages in concurrent execution of activities. Based on empirical analysis, the authors find that overlapping actually leads to an increase in development costs if stage overlaps are not planned carefully. This result is particularly relevant to the COD structure of P_X , since its overlap amounts are estimated early in the setup phase (see next section). For example, if downstream activities that depend heavily on upstream information are started early, then major iterative rework may ensue if some of the prereleased upstream information is found to be incorrect or inaccurate in later downstream stages. It is, therefore, necessary to first identify the individual stages of tight coupling before a full-scale overlapping endeavor can be formally invoked. This guideline also influences the management of P_X . Similar results were found by Eastman [5], who investigated the practice of engineering information release prior to final design freeze. The author recommends that the practice should be reserved only for situations where it is known with certainty that the released information is complete, accurate, and unambiguous.

In studies on the performance of concurrent engineering in commercial product development processes, Imai *et al.* [13] and Takeuchi and Nonaka [3] reported that overlapping of engineering activities always produced faster development. The authors compare this to a game of rugby, to the effect that the entity evolves out of the constant interactions of a specialized and multidisciplinary team of players who work closely with one

another from the beginning to the end of one or more stages. Such complex activities have a major impact on product design itself. Other researchers [10], [14] have further investigated this specific aspect of the problem. For example, concerning manufacturability that involves both product as well as process design, each is usually carried out by separate, often distributed, teams of engineers. The main benefits here are *parallel development*, in which product design reviews provide information enabling process design to become mature at an early stage. Also, *product quality control*, in which the validity and applicability of a design is revealed through multiple design reviews, facilitates early detection of flaws and errors. Allowing for a certain setup time associated with each review, a balance must be attained between too frequent and too infrequent reviews in order to achieve both time shortening and cost reduction simultaneously. A similar conclusion is reached in the work of Smith and Reinertsen [1], who recommend that overlapping should be carried out through the exchange of “partial information” between co-evolving processes; thus, downstream activities benefit from early-released information, accelerating overall product development. Some deficiencies in the process are pointed out by the authors.

Smith and Reinertsen do not provide details as to how to specifically deal with the problem of downstream rework caused by the incomplete nature of the early-released information. This issue is addressed by Krishnan [7] and Krishnan *et al.* [4]. Adopting an information-processing view [15], [16] of product development, the authors studied the overlapping of activities using a model-based framework. They emphasize the importance of careful management of the overlapped product development process and recommend that: 1) “the effect of overlapping on development performance should be used as the basis for disaggregating the exchanged information” into its various parts; 2) “only those parts whose early freeze would produce very little quality loss should be finalized (frozen) early”; and 3) “the upstream information exchanged in a preliminary form should be chosen such that changes in its value may be absorbed without a substantial increase in the downstream effect.”

Admitting the general dynamic complexity of the COD endeavor, several researchers [8], [17], [18] considered it wise to limit the practice to short-term projects of low risk and uncertainty determined by various external sources (for example, market demands, fluctuations in the availability of resources, etc.). In this context an efficient way to minimize undesirable downstream rework is to try to reduce uncertainties and conflicts in upstream development as early as possible [3], [4], [19]–[21]. Thus, an ordinary, low-risk software product of limited scope that is in a hurry to reach the market or to avoid a quick decay into obsolescence could benefit from COD in an effort to attain a faster-than-average velocity of progress. However, P_X 's objective is different. It seeks to produce an innovative software product based on a well-chosen group of subsidiary products as major R&D endeavors with longer-than-average time horizon of development. Here, uncertainty resolution proceeds primarily through continuous down-the-line iterations of work and rework; the rush for “time-to-market” is practically nonexistent.

Much of the COD research has focused on problems of large-scale industrial manufacturing [22], aerospace and aeronautical engineering [for example, work at the National Aeronautics and Space Administration (NASA)/Jet Propulsion Laboratory (JPL)] [23]–[25], and Internet-based businesses involving interdisciplinary applications [26]–[29]. An overall review of the existing research literature makes it clear that there seems to be no general, all-encompassing way of applying COD techniques to an arbitrary engineering enterprise. Each application seems to be unique in its own way, involving many subtleties and problem-specific peculiarities. However, a number of general results and recommendations can be shortlisted to apply to a group of practical engineering activities, including some forms of software development in IT [30]–[32]. There is, however, scanty evidence of any actual research on the use and application of the above ideas in software R&D. This is primarily the area in which the present work makes its contribution by utilizing the specific model of P_X .

III. OUTLINE OF THE SOFTWARE ENGINEERING METHODOLOGY OF P_X

It is fairly well known that many IT projects fail when they become excessively complex [33]. Concerning software engineering practices of COD projects, the question that naturally arises is, how one actually copes with the high level of dynamic complexity inherent in these projects. Using the example of P_X , we, in this section, throw some light on the issue. However, as already mentioned, the software engineering of P_X is a representation of a set of common COD methodologies derived from two independent, industrial IT projects. Only those aspects of these projects that are found to involve a similar framework of processes and procedures adapted to COD are included in P_X . Other features in which the projects differ in various ways are primarily attributed to their individualized demands and of the concerned businesses. These are not particularly relevant to this study, and we, therefore, leave them out. The COD practices of P_X that we now proceed to describe will provide a background for the analytical model elaborated in the next section. Our discussion here focuses on three principal aspects of P_X : 1) the initial project setup functions; 2) the processes of conflict resolution in integration management; and 3) component coordination and synchronization.

A. Initial Project Setup Functions

As described in the introduction, P_X is developed by appropriately combining development output from its overlapping components (the C_i 's) selected in $\mathcal{G}_X$. Its early stage embodies the initiation and setup processes to specify upfront the overall design constraints. If performed well, it results in fewer downstream problems, such as conflicts between component teams, project milestone delays, costly rework, etc. During this phase, a number of software engineering activities are undertaken, with a group of additional subactivities and subtasks involved in some of them. In the COD framework a number of these processes must proceed simultaneously [34], forming feedback systems, whereby one process depends for its outcome on the decisions

made in a related, coevolving process. The most important activities are the following.

- 1) Identify all the C_i 's together with any other collaborative technologies needed in P_X .
- 2) Ascertain as completely as possible their primary scope, flexibilities, and compatibility issues.
- 3) Perform an initial assessment of the requirements of P_X by means of critical comparisons of all the functional requirements of the C_i 's together with associated constraints. This customized process is developed iteratively by means of the following subprocesses:
 - a) refine the description of the project compatibility identification scheme into a set of patterns to delineate project integration activities;
 - b) further refine them into more detailed task patterns for each C_i .
- 4) Review all high-level objectives with the concerned management.
- 5) Proceed with formal approval and documentation.
- 6) Form $\mathcal{G}_X$. Identify its domain and the different functions that it should support with a more thorough analysis of the informational and functional areas of each C_i , as obtained from the previous process activities.
- 7) Form an assessment of the system boundaries of the C_i 's and the magnitudes of initial overlap between them, in the capacity that these components will participate in COD within the controlling domain of $\mathcal{G}_X$.
- 8) Obtain approximate numerical estimates of the initial dependencies. Examples of some real indicators that were actually used in the projects include estimates of an anticipated number of API calls between two dependent components or the number of object callbacks in an event-notification design between modules belonging to different components as determined from Unified Modeling Language (UML) interaction or component diagrams.
- 9) Attempt to reduce the dependencies of some of the C_i 's. In practice, this step is carried out by employing an iterative trial and error procedure as follows.
 - a) Determine a threshold dependency level between the C_i 's. In practice, measures of information overlap between previous projects have sometimes guided the procedure for determining the threshold. Ordinarily, the threshold is the minimum possible amount of coupling between two activities in dependent components. This value, which is based on preliminary design considerations, is commonly obtained by estimating *in advance* the number of instances of upstream information that need to be exchanged between the activities over a period of two weeks since inception.
 - b) Identify a subset of the C_i 's whose mutual dependencies already seem to fall below the threshold.
 - c) Redefine requirements, functionalities, and scope of these components.
 - d) Finally, eliminate the dependencies completely.

Henderson and Clark [35] specify that one of the biggest challenges in complex technology development and innovation

is making the overall architectural dependencies transparent at an early stage. Thus, as the number of dependencies increases between the components, the complexity and the amount of development effort to share and assess mutual knowledge at the boundaries of the components also increases. At the end of a successful setup phase, one should, therefore, expect to see a $\mathcal{G}_X$ controlling fewer strongly coupled components. Some of the complexities associated with downstream integration activities will thus have been effectively liquidated.

B. Uncertainty and Conflict Resolution in Integration Management

After the operations in the setup phase have been dealt with satisfactorily, project development activities are started in P_X and the C_i 's in an overlapped fashion. Uncertainties and conflicts subsequently appear in the requirements, design, implementation, and quality assurance procedures. Besides, the specific constraints imposed by P_X itself on the C_i 's limit their ongoing development. However, the components selected in $\mathcal{G}_X$ are such that their design and development activities are similar. For example, all projects are required to follow the organization's project development guidelines, satisfy specific implementation standards, use prebuilt, tried and tested library and API methods, make consistent use of similar or identical technologies, and so on. The expectation is that, each C_i should function self-sufficiently and control the internal complexity restricted to its own functional domain. The *global* dynamics of P_X should, therefore, remain vastly unaffected. In order to ensure that a smooth integration process exists between the C_i 's, the following activities are carried out.

- 1) In upstream development in any of the C_i 's, identify the specific requirements that are in conflict with one another. Do not release any information at this stage.
- 2) Using quality procedures resolve initial uncertainties and conflicts in the requirements of the concerned C_i 's.
- 3) Modify these requirements as necessary to accommodate those of P_X .
- 4) For overlapping and independent multistage operations, start concurrent design at this stage and release preliminary upstream information to activities in concerned components to aid in their downstream development.

Concurrent design endeavor [36] involves different kinds of preliminary design for major coordination between the C_i 's. It is also commonly observed that the interdependent design decisions released upstream frequently result in design rework downstream in which the design change leads to alterations of previously implemented modules, which are periodically absorbed during integration.

In this context, two parameters introduced by Mitchell and Nault [37] can be used in P_X to keep track of local fine-tuning of overlap integration activities in the components. The first is a *frequency dimension* that gives the number of change iterations. The second is a *magnitude dimension* that measures the total amount of change relative to the original design. Rework in P_X in multiple stages of iterations leads to *project delay*, which is the time lag between planned and actual project completion [37].

To alleviate problems of delay, the design modifications of P_X and the C_i 's are made as nondisruptive as technically allowable. However, once $\mathcal{G}_X$ has been put in perspective, this task does not in practice pose a major problem. The reason for this is that, since P_X is larger in size and scope than each of the C_i 's, it can be made considerably more flexible and relaxed in its requirements and the resulting implementation. It also has a longer time horizon in R&D, so that project risks are vastly mitigated by continuously reworking and reengineering the final product for maturity.

In order to determine which activities can be finalized early and which can be released in some preliminary form, two characteristics of the development process are of importance [4].

- 1) Upstream information evolution is used to refine the upstream generated information from its preliminary to the final form.
- 2) Downstream iteration sensitivity yields the relationship between the duration of downstream work and the amount of change in the value of the corresponding upstream information.

To streamline the overlapping process, two recommendations [37] are put into practice in this connection. First, upstream activities are evolved at a rapid pace, whereby the information to be released is consolidated into a final form quickly, but without much loss in quality, and then frozen. It is subsequently passed downstream on to a dependent activity. This process is called *fast evolution*. Second, and also simultaneously, the characteristic of *low sensitivity* is observed, whereby significant changes in the information used in downstream work is quickly absorbed by the downstream activity. Specifically, a slight variant of the above procedure was successfully implemented in one scenario as follows.

- 1) First, freeze out a fixed amount of upstream activities in every cycle of development of a component.
- 2) Next, release this unit downstream to one or more activities in a dependent component through fast evolution.
- 3) Observe the low-sensitivity criterion and ensure that the iteration frequency dimension is tolerably small.
- 4) Finally, absorb the received input in the dependent component.

Note that no direct engineering is performed on the received input. Also, only its impact on the downstream output is analyzed when evaluating the frequency dimension. Should inconsistencies or ambiguities arise, a minimalist approach is adopted to eliminate conflicting factors during integration. Good communication practices between concerned development teams have proved to be of great value in this regard.

C. Component Coordination and Synchronization

Project task coordination in concurrent engineering activities involving multifunctional teams is extremely crucial to guarantee a smooth evolution of the project [38]. Some organizational barriers to establishing proper coordination include unfriendly or hostile work environment, lack of proper organization, improper communication, poor consistency, and presence of imbalance in the management of important links [39]. Frequent, bilateral communication and early release of information at se-

lected stages of development can considerably improve the state of coordination activities [21], [40]. Krishnan *et al.* [4] recommend that the information exchange method should be specifically geared toward the actual characteristics of the exchanged information. For example, to avoid excessive, unanticipated iterations [41] in component coordination, only some types of information can be directly used in preliminary form. In P_X , the following initial coordination activities are instituted.

- 1) Before the actual integration takes place, frequent formal as well as informal intercomponent communication (for example, face-to-face and electronic exchanges, developer sessions, emails, biweekly meetings with managers, etc.) is started.
- 2) All outstanding issues and points in need of resolution or clarification are promptly communicated to the concerned requirements teams.
- 3) The necessary documentation of these issues is carried out.

Next, more specific coordination efforts are expended during the actual integration phases. These include, at a minimum, the following.

- 1) As needed, the requirements, visioning, design, specific implementation, and quality assurance procedures of each component are modified.
- 2) All modifications are initially confined to the level specific to a single, affected component, producing only local downstream iterations in it.
- 3) It is strictly observed that the primary business goals of that component are not grossly altered.

In summary, the requirements and design modifications must first be carefully coordinated within the functional domain of each component. These activities are collectively termed the *synchronization points* of COD. An integration assessment, performed at every synchronization point, involves the following multistep procedure.

- 1) Review all planned processes to gauge the impact of the modifications on them.
- 2) Baseline project deliverables after assimilating the new changes in requirements, design and implementation.
- 3) Apply the adjustments and modifications and keep track of the current frequency dimension.
- 4) Prepare for the next phase of development and subsequent integration.

To smooth away problems of coordination at any synchronization point of P_X , requirement and design analysts from all the components prepare and analyze lists of conflicting requirements, along with related constraints imposed by COD. To obviate local conflicts and inconsistencies, the concerned software engineers search for acceptable alternative solutions or workarounds. The architects and designers of the affected components are then brought in to develop ideas and work toward mutually beneficial solutions. It is important to note that component coordination at every synchronization point should also complete within a reasonable time frame, since a delay could spell uncertainty in multiple downstream development efforts, caused by excessive iterative rework [4]. Theoretical models [42] predict that strong coordination activities are needed to

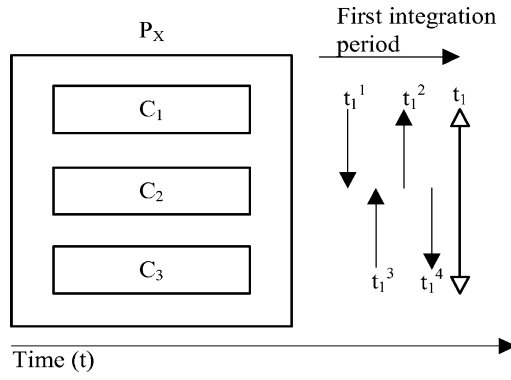


Fig. 2. Schematic diagram of P_X with overlapping components.

accomplish early resolution of uncertainty. It also makes design and requirements stable and minimizes project risks and delay [20].

IV. DYNAMIC SYSTEMS ANALYSIS OF P_X

In this section, we present an analytical model characterizing the asymptotic dynamics of concurrently overlapped software projects of the type exemplified by P_X . To handle a relatively simple scenario for describing short-term evolution for which data are available, we restrict our model to analyzing first-order behavior only. Although this is a less accurate description, we see that even this limited model can provide some valuable information about long-term stability. The model is of theoretical interest too, as it also serves as the first step toward understanding many other complexities of COD that appear only in higher orders of component integrations. Before moving into technical details, it will be useful to briefly describe the basic terms and definitions used in the present analysis. The discussions presented here are self-contained; for a more in-depth study, references are provided.

A. Theoretical Concepts and Preliminaries

The methodology of dynamic systems analysis allows us to conceive of P_X as a vector with a fixed number of components in a certain vector space $\mathbb{V}$ called the *project space*. These components, commonly known as the *state variables* of P_X , symbolize the individual projects that are controlled by COD within the project group $\mathcal{G}_X$. The project vector satisfies a time evolution equation, tracing out a trajectory in $\mathbb{V}$. This equation is usually given in terms of a *project operator* α that determines the temporal behavior of the individual state variables. A study of the characteristics of this trajectory in $\mathbb{V}$ provides valuable information about the long-term sustainability, maintenance, and management of the COD project.

Fig. 2 illustrates a simple COD interaction scenario involving a P_X that consists of only three components C_1 , C_2 , and C_3 interacting in overlapped upstream and downstream development activities. Several such activities are likely to be concurrently deployed in the components. Not all paths of these activities need to exchange upstream information as development progresses, but some inevitably do. For example, in order to display in a

window on the screen a set of relevant inventory data concerning sales orders of laptop computers processed over the last ten days, the associated GUI component makes a call, say, to a database through a socket interface made available by an intermediate, middle-tier server application. Although the general window design in the GUI component can proceed independently at least in this scenario, the data retrieval process must obtain preliminary upstream information from the middle-tier component as to the general nature of the interface of the call. Similarly, the design of the middle-tier component requires preliminary upstream information from the database component concerning the request-response transaction between the two. Such information may be premature at the time when they are sought, and any subsequent changes in the information (in the call interface only, not in the internal implementation details) will give rise to downstream modifications.

As shown in the figure, at time $t = t_1^1$, C_1 releases upstream information to C_2 . The information produces local downstream modifications in C_2 , which it subsequently absorbs. If conflicts arise due to the nature of the received information, C_2 sends its feedback to C_1 at some time $t = t_1^2$, based on which C_1 adjusts its downstream work and modifies the previously dispatched information and releases it again to C_2 at a later time. A similar interaction takes place between C_3 and C_2 at the times t_1^3 and t_1^4 , where t_1^3 is intermediate between t_1^1 and t_1^2 . Thus, over the interval $t_1^2 - t_1^3$, C_2 overlaps simultaneously with C_1 and C_3 . Finally, at $t = t_1$, where $t_1 > t_1^4 > t_1^2 > t_1^3 > t_1^1$, all development activities in the components are synchronized, and a major integration is performed. The time t_1 is considered as one integration period of overlapping development. The relevant development output from the components along with related activities in P_X are consolidated into P_X at this time, constituting one incremental unit of temporal progress. Over the entire duration of the project, the integration happens at repeated, fixed periods. Actual distributive overlapping is practiced in the R&D environment when P_X is developed. In this scenario, although some components may exhibit large static dependencies, their information sensitivity at the design level is kept as low as possible. Further, while the upstream work generally evolves at a rapid pace, the downstream activities begin with some form of preliminary information and absorb design modifications subsequently in multiple iterations over time.

To be completely general, we represent, in the mathematical model, P_X by a project vector $\vec{P}$ in an N -dimensional representation $\mathbb{V}^N$, which consists of $1, \dots, N$ component projects. Quantitatively, $\vec{P}(t)$ embodies the net output of development in P_X completed at an arbitrary integration period t , and the output amounts from the components at this time are denoted by scalar variables $p_1(t), \dots, p_N(t)$. In vector notation [43], [44], $\vec{P}(t) = (p_1(t), \dots, p_N(t))$. Let us denote the k th period by t_k that represents a specific discrete value of the time. Variable $k \in \mathbb{I}^+$ is an indexing variable of the positive integers $\mathbb{I}^+$.

Next, we consider the following three fundamental resource units that contribute to every component of P_X .

- 1) H : It is the human contribution resource that stands for innovation capabilities, work efficiencies as well as the

levels of technical, social, and communication skills of the human players.

- 2) S : It stands for the technology contribution resource that models the organization's existing information systems (IS) infrastructure and facilities involving all software and hardware tools and IT services available to a project together with their overall utilities and resourcefulness.
- 3) K : It is the knowledge resource that quantifies the amount of explicit project development knowledge assimilated as a result of the frequency of information exchange instances between coevolving components.

In the next section, we present a simulation of P_X in which examples will clarify the significance of the above resource units. For now, simply view them as predictor variables in the mathematical formulation of the problem.

B. Mathematical Formulation

The distribution of the resources in any component can be described by a completely general, inhomogeneous quadratic form

$$p_i = \sum_{a,b} \rho_{ab} R_{ia} R_{ib} \quad (1)$$

where ρ is a resource coefficient matrix, and R_i is a resource unit proportional to H_i , S_i , or K_i , and p_i is the net output in component C_i . Here, a and b represent matrix indices. In most common situations, the resource units can be taken to be independent predictors of the dependent variable, for which ρ is a real, symmetric matrix. In that case, the quadratic form can be diagonalized by an orthogonal transformation, which, after a rescaling, yields the homogeneous equation

$$p_i = H_i^2 + S_i^2 + K_i^2. \quad (2)$$

We take this as the *defining* equation for p_i , with the understanding that, in any software engineering project, human, technology, and knowledge resource variables constitute the fundamental inputs to the project at all times. For a fixed $p_i > 0$, (2) defines a *spherical distribution*. This particular form is useful, and it is also the simplest¹ over a spherically symmetric manifold of resources. It is natural, therefore, to have the form (2) available at all times to every component of P_X . In a real situation, some deviations may be observed if projects are monitored for a sufficiently long time. In that case, the original form (1) should be employed. In this study, we will be content with the conceptually simpler case. For the time span over which the present model has been studied, we also do not see major deviations from homogeneity that are interesting enough to be reported.

The project evolution can be represented in the project space (mathematically, the phase space) by a general vector differential equation of the form

$$\frac{d}{dt} \vec{P}(t) = \alpha(t) \vec{P}(t). \quad (3)$$

¹It is possible to use other functional distributions in a simulation; for example, a linear form could be used as a special case in which the resources would be distributed on a plane considered as a degenerate spherical manifold.

Time is not a “coordinate” here but a parameter that can be chosen appropriately to suit the specific dynamics of the problem (as mentioned later). In order to follow the development of P_X based on actual observational evidence, we are interested in discrete-time, autonomous simulations, for which the most general discrete-flow version of (3) can be obtained by taking its discretized time limit, which yields the general linear, inhomogeneous equation

$$\vec{P}(t_{k+1}) = \alpha \vec{P}(t_k) + \vec{Q} \quad (4)$$

where $\vec{Q}$ embodies the cumulative effects of all temporally non-varying sources in P_X . Since these are identical at all times, the inhomogeneity term is of no practical import and can be dispensed with by the linear transformation $\vec{P}(t_k) \rightarrow \vec{P}(t_k) - (\mathbb{I} - \alpha)^{-1} \vec{Q}$, where $\mathbb{I}$ is the identity operator (matrix) having the same dimension as α . This transformation converts (4) to the homogeneous form

$$\vec{P}(t_{k+1}) = \alpha \vec{P}(t_k) \quad (5)$$

which is better suited to our simulations. The continuous-time flow dynamics can also be studied, if so desired, in the project space by analyzing the continuum limits of the discrete version [45]. Note that both dynamical equations are fully deterministic. Instead of directly incorporating stochastic distributions as additive disturbances in these equations, we have chosen to embed the stochastic structure directly into the procedures used for determining the parameters of the model, which is a common practice in many econometric models [46], [47].

As evident from equation (5), $\vec{P}(t_k)$ depends both on its initial value $\vec{P}(t_0)$ as well as on the project matrix α . This matrix is a $\mathbb{N}^N$ representation of the project operator, generating a new unit of $\vec{P}$ from the previous unit at every specified integration period (a suitable measure of the time-scale parameter). It is the elements of this $N \times N$ matrix that determine the local as well as the long-term dynamics of the project. Variable α has the general form

$$\begin{pmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1N} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{N1} & \alpha_{N2} & \dots & \alpha_{NN} \end{pmatrix}. \quad (6)$$

The parameters of our model are encoded by the dimensionless, positive measures of integration overlap given by α , which is real, square, and positive-semidefinite. Its diagonal elements represent the magnitudes of self-integration. Ideally, these should be unity, but in the actual projects some deviations were observed. Furthermore, on the basis of observational data, we also restrict α to be symmetric ($\alpha_{ij} = \alpha_{ji}$). Thus, the strengths of the overlaps between conjugate pairs of components are equal. The computational advantage of a symmetric matrix is considerable; it reduces the number of independent components from N^2 to $(1/2)N(N+1)$, which, for moderately small N (as in P_X), is a substantial saving in computational time. The use of this constraint is justified by the observation of a similar trend in the actual data that reveal simply a small difference in the values of the transposed pairs of the matrix of the order of ± 0.2 on a scale of 0 to 1. However, this could well have been

due to a particular characteristic of P_X , and it is certainly not to be expected, in general, of all COD projects. For example, in situations where the overlap data for the transposed pairs show differences, say larger than 0.5 in absolute units, the use of symmetry in the simulation is unwarranted. The dynamics of such an unconstrained system requires more general eigensystem evaluation procedures, with additional demands for computational time and resources. This, nevertheless, is merely a computational issue; it does not alter the physical implications of the results of this study in any way.

The linear, homogeneous, and autonomous [48] evolution equation (5) admits of a solution in the general form

$$\vec{P}(t_k) = \alpha^{t_k} \vec{P}(t_0) \quad (7)$$

where $\vec{P}(t_0)$ is a fixed, initial vector of the state variables, which, in practice, is determined by means of a suitably chosen statistical procedure (next section). Notwithstanding the simple appearance of (7), its direct, practical implementation in an actual simulation is very difficult, especially when α is high-dimensional.² In anticipation of this difficulty, we use a technique that obviates the need for tackling the eigensystem in its entirety. Moreover, the technique is specifically adapted to investigating the asymptotic dynamics in which we are particularly interested. Before going into the details, let us first derive a solution of the eigenvalue problem in a form that is conducive to numerical simulations of P_X .

Consider the N -dimensional project space $\mathbb{V}^N$ of $\vec{P}$. The project operator α acting on $\vec{P}(t) \in \mathbb{V}^N$ generates a trajectory in time in $\mathbb{V}^N$. A matrix representation of α given by (6) in $\mathbb{V}^N$ corresponds to the most general, arbitrary orientation of basis vectors of $\mathbb{V}^N$. However, there exists in this space a set of principal axes, such that if basis vectors can be chosen along these axes, the representation (6) can be diagonalized. Since α , in our choice, is a real, symmetric matrix, the principal axes rotation can be performed by means of an orthogonal transformation, such that the surviving elements of α are just its eigenvalues, with the corresponding eigenvectors constituting the basis vectors oriented along these axes. The rationale behind this procedure is that, an eigenvalue is needed to construct a suitable, asymptotically stable project vector in the evolving dynamics. Suppose then that $\lambda_1, \dots, \lambda_N$ are the eigenvalues of the overlap eigensystem. They may not necessarily be all distinct; if this is indeed the case, then there may not be enough orthonormal eigenvectors in $\mathbb{V}^N$ to perform the desired axes transformation. This problem is technical, but can be dealt with by a slight modification of the present formulation. However, its final physical implication is exactly the same as that of the distinct-eigenvalue scenario. Thus, in this case, α is truly nonsingular, and a sufficiency condition guarantees that it has N

linearly independent, nondegenerate eigenvectors $\vec{V}_1, \dots, \vec{V}_N$. We may now express the initial $\vec{P}(t_0)$ as a linear combination of the $\vec{V}_i$'s as

$$\vec{P}(t_0) = \sum_{i=1}^N c_i \vec{V}_i \quad (8)$$

where each $c_i \in \mathbb{R}$ is a real constant that depends on the initial values specified in $\vec{P}(t_0)$. Noting that $\lambda_1^{t_k}, \dots, \lambda_N^{t_k}$ are the eigenvalues of α^{t_k} , we obtain from (7) and (8) the complete solution

$$\vec{P}(t_k) = \sum_{i=1}^N c_i \lambda_i^{t_k} \vec{V}_i. \quad (9)$$

This allows us to write down an asymptotic stable extension

$$\vec{Q}(t_k) = \lim_{t_k \rightarrow \infty} \frac{\vec{P}(t_k)}{\eta^{t_k}}. \quad (10)$$

Here, η is the spectral radius [49] of α and is defined as $\eta = \max_{1 \leq i \leq N} |\lambda_i|$. Note that $(\lambda_i/\eta)^{t_k} \xrightarrow{t_k \rightarrow \infty} 0$ for $i \neq m$, where the eigenvalue λ_m corresponds to η . In most situations, λ_m is nondegenerate, being a unit multiplicity root of the eigenvalue equation. It is necessarily a real number, since complex eigenvalues occur in conjugate pairs having the same modulus. Equations (9) and (10) can now be combined to produce the asymptotic solution

$$\vec{Q}(t_k) = c_m \vec{V}_m + \sum_{i \neq m} c_i \left(\frac{\lambda_i}{\eta} \right)^{t_k} \vec{V}_i \quad (11a)$$

$$\sim c_m \vec{V}_m \quad (11b)$$

where the asymptotic limit is symbolized by $\sim$. Interestingly, (11b) provides an estimate for the eigenvector $\vec{V}_m$; thus, if we know λ_m , we can easily approximate $\vec{V}_m$ with $\lambda_m^{-t_k} \vec{P}(t_k)$ for some large t_k . This is indeed a very useful result in both analytical as well as computational work, since the asymptotic behavior can be described solely in terms of a *single* eigenvector—the only one corresponding to the spectral radius of α ; the other eigenvectors simply fade into insignificance in this limit, and consequently, save us the trouble of having to find them individually. Of course, the approximate eigenvector is known only up to a constant multiple, but this is of no concern, since the multiplier can be disposed of by a suitable normalization prescription.³

The final task then is to find a suitable estimate for the spectral radius η . As before, it is difficult to directly compute this quantity in closed form, but we can again find a valid numerical approximation for it in the asymptotic regime. First, note that the Euclidean norm of $\vec{P}$ is given by $|\vec{P}| = \left(\sum_{i=1}^N p_i^2 \right)^{1/2}$, since $(\lambda_i/\eta)^{t_k} \xrightarrow{t_k \rightarrow \infty} 0$ for $i \neq m$, where, as before, λ_m corresponds to η , the desired approximation is given by the quantity

²Reducing the dimensionality of the eigensystem of the project matrix is, to a certain extent, within our control and is an important task for the project modeler. Often, in a real project, there may be a large number of components to contend with; however, in order to simplify the modeling process, it may be possible (with the aid of additional constraints) to consolidate several related components into a single one; this serves to reduce the dimensionality and simply the solution. It is, therefore, expedient to take advantage of this method in a practical situation.

³Note that eigenvectors are not unique; a scalar multiple of an eigenvector is also an eigenvector.

(see the Appendix for a proof)

$$\eta \sim \frac{|\vec{P}(t_{n+1})|}{|\vec{P}(t_n)|} \quad (12)$$

where t_n is fairly large (in numerical work $n \geq 15$ will suffice). The computational scheme is to evolve an initial $\vec{P}(t_0)$ using (5) in order to arrive at an approximation for η from (12) for some sufficiently large t_n . Having obtained estimates for η and $\vec{V}_m$, it is now possible to relate the resource amplitudes to the ratios of the components of $\vec{Q}$. For a given value of the integration time, (2) represents a 2-D constraint surface embedded in a 3-D real manifold of the resources. The time evolution of this surface is dictated by (5); therefore, for every unique value of the period, the surface moves from its initial configuration to a new configuration corresponding to that value of the period. Since the dynamics is linear and the distribution of resources is homogeneous at all times, the constraint equation remains form-invariant under time translation. Equations (2), (5), (11b), and (12) are then the key results of the present model that allow us to correctly identify the domains of the parameters for stable asymptotic behavior of P_X . In the next section, a numerical simulation of P_X illustrates the use of these equations to understand the nature of the long-term stable dynamics. This type of analysis is very useful in successfully managing complex COD projects over a period for which the present first-order analysis is viable.

V. COD SIMULATION

Based on the analytical results derived in the previous section, we now present a numerical simulation to illustrate the long-term evolution and stability of P_X with its stages of overlapping development activities. The simulation is based on a simplified, small-scale version⁴ of the actual projects whose common COD processes are consolidated into the model of P_X introduced previously. We first introduce the projects, the organizational culture and the technological environment, then describe the underlying method of data collection and analysis, and finally discuss the simulations and the results.

A. Project, Organizational Culture, and Environment

This study evolved with the author's personal involvement in two similar COD projects in the IT industry. The projects were undertaken in R&D by two independent, large business organizations, the first one in the U.S. telecommunications sector, and the second one in IT software product services. Both companies were concerned with product innovation in IT software applications using some established along with some experimental procedures to carry out the overlapping development.

Some form of personal as well as organizational learning culture was observed to have been maintained in all of the product development phases. It is, therefore, logical to assume that the

human contribution is incrementally enhanced by the different skills and experiences gradually acquired by the team members while working on their projects. Some new engineers and a few independent consultants were hired during intermediate stages of P_X , and the overall employee retention rate was found to be high. As regards technological resources in software development, uses were made of modern operating systems, networking facilities, requirements management tools, scalable modeling and design software applications, programming resources such as compilers, debuggers, profilers, and sometimes, complete commercial IDE's, automated testing and performance analysis tools, defects-tracking systems, etc. While the details of the design and implementation processes pertaining to the upstream-released information were found to change downstream in most instances in overlapping development, the process abstraction to which the specific information relates were controlled so as not to change drastically over successive downstream iterations. C_X employed an IS infrastructure strategy to plan, organize, and manage the level of knowledge in these iterations. Process asset libraries and repositories were some of the means through which information and learning were shared among project engineers and stakeholders. Artifacts, such as upstream process and method descriptions, downstream design and implementation plans, directions and standards, etc., were provided for adaptation, modification and reuse. Also, a WIKI was extensively used to share and enhance project information as well as to provide adequate technical references. White papers and project-related documentation were meticulously maintained in all phases of development.

B. Data Collection and Study Methodology

In the present simulations, no data from the actual projects are used that belong to the proprietary domains of the software product developed by an organization. These include the internal details of the product innovation strategies, requirements specifications, designs of software applications, source code, etc. Only aggregate-level data are statistically processed in the simulations. Also, approximately the initial seven months of data that are relevant to the present simulations have been utilized. After about eighteen months of continuous product development, one of the companies outsourced parts of its projects to two offshore vendors and reallocated a number of COD assignments to one of its offshore captive units. Data from these domains also lie outside the scope of this study.

Project data are employed in the simulations in a number of ways. First, numerical values for the parameters of our model are obtained as regression estimates from the data. Second, the data are used to examine simulation results for model validation and accuracy. Finally, many synthetic datasets in the Monte Carlo simulations of our model are directly generated from the actual data. The primary sources of project data include managers, developers, quality assurance engineers, requirements and business analysts, etc. Published and unpublished internal project reports, reviews, spreadsheets, meeting minutes, and general guidelines provide additional supportive evidence for raw data.

⁴We continue to designate the project used in the simulation by the symbol P_X , although we must keep in mind that this P_X is actually a small-scale version of an actual industrial project, and is therefore somewhat simplified and adapted to suit the simulation.

Following are brief notes on the survey and data collection methods used in this study.

- 1) *Personal interviews and surveys*: Primary samples of data were obtained by direct, face-to-face interviews of software engineers individually and sometimes in groups of two to three. Development and project managers of P_X and its components were interviewed individually. Data concerning project goals, targeted processes of component integration activities, estimated and actual completion dates, estimates of overall COD progress, approximate overlap dependency measures, estimated averages of component resource distribution, and project delays were collected through a combination of oral interviews and written surveys.
- 2) *Examination of documents*: Published and unpublished internal project reports and reviews were studied to validate survey data wherever feasible and also to procure additional information relevant to the projects. The approximate validity of data obtained from various reports, white papers, and WIKI pages was confirmed by cross-checking them against estimates provided by software engineers and project managers.
- 3) *Collection and examination of data*: Attempts were made to reduce bias in the data obtained from group interviews by including people with heterogeneous sets of skills (for example, developers and testers; requirements analysts and design architects, etc.). Also, in collecting data on component integration and the corresponding synchronization points, the same engineer was not interviewed in two successive integration cycles. Since approximately 40 h of product development activities constituted one period of integration in P_X , we assumed a time scale equal to three integration periods to be a reasonable frame for erasing some of the “memory” of the data provided in a previous interview. Care was also taken to make each survey query as independent and self-contained as possible. In every team of software engineers, the data provided by the team leaders were independently validated against corresponding estimates provided by the general members. In a few instances, discrepancies were noted. In situations where it was not possible in any way to locate or identify the sources of these discrepancies, the concerned data points were expunged from the sample sets. In a few other instances, some itemwise nonresponses were encountered. These were, for the most part, due to the respondent’s lack of domain knowledge of the underlying system at the time of a specific interview. Finally, it is interesting to report that, on two rare occasions a seemingly disgruntled software engineer provided questionable data on integration rework iterations. These values were gross outliers, bearing little or no resemblance to data collected from other sources and would produce an unnecessarily large skew in the regressions. These were also omitted from the analysis. Matched sampling techniques were used when the same persons were interviewed before and after component integration and the associated project synchronization points. Activities in the project

setup phase were recorded accurately, where managers of the components, along with the managers of P_X , provided measures of downstream uncertainties, component scope, implementation horizon, and other estimates of collaborative and cooperative planning utilized in the component integration activities.

C. Small-Scale P_X —A Simulation

We discuss below the following topics pertaining to the COD simulation of P_X .

- 1) Details of P_X used in simulation.
 - 2) Project parameters, data and the simulation method.
 - 3) The integration-time evolution and model validation.
 - 4) The component output ratios and model validation.
 - 5) Two-mode projections of resource distributions, phase diagrams, and their use in project management.
- 1) *Details of P_X* : The small-scale COD project P_X used in our simulations combines the following five components in $\mathcal{G}_X$.
- 1) C_1 develops a standalone, backend server software application running on a Linux platform. It supervises a number of subsidiary business processes that serve specific client requests through an XML interface. The programming languages used to code the application are C++ and Java.
 - 2) C_2 develops a database software application in ORACLE running on HP-UX that stores and manipulates client data. The application also provides a GUI module, which is used for the purpose of storing and modifying business data.
 - 3) C_3 develops a proprietary GUI software application with supporting database and reporting facilities. The project employs .NET technology, with the coding done in C# and VC++.
 - 4) C_4 develops a middle-tier, CORBA-based software application offering a wide range of business functionalities.
 - 5) C_5 develops a group of collaborating network software applications that ensure correct communication over various types of networks and protocols between different applications developed by C_X .
- 2) *Project Parameters, Data, and the Simulation Method*: As discussed in Section IV, we can define a general resource vector of the form $\vec{C} = H\hat{h} + S\hat{s} + K\hat{k}$, where $\hat{h}$, $\hat{s}$, and $\hat{k}$ represent, respectively, the human, technology, and knowledge units in $\vec{C}$. The development output at integration period t is related to the square of the Euclidean norm of $\vec{C}$: $p(t) \propto |\vec{C}(t)|^2$. The starting level of project output for each component is obtained from the spherical manifold of resources characterized by (2) evaluated at $t = 0$. In order to obtain suitable estimates for the starting amounts $p_i(0)$ needed in the simulations, we use the following set of production functions:

$$H = H_{\text{int}} X_h^\alpha Y_h^\beta Z_h^\gamma \quad (13a)$$

$$S = S_{\text{int}} X_s^\alpha Y_s^\beta Z_s^\gamma \quad (13b)$$

$$K = K_{\text{int}} X_k^\alpha Y_k^\beta Z_k^\gamma \quad (13c)$$

In this model, the average level of human output H is assumed to depend on three⁵ contributory variables:

- 1) technical skills and innovation capabilities (X_h);
- 2) effective hours of daily productive work (Y_h);
- 3) communication and social skills (Z_h).

Similarly, the average level of technology output S is assumed to depend on three factors:

- 1) utility or usability (X_s) of a specific technology resource (for example, a compiler optimized for a certain CPU architecture);
- 2) purchase price (Y_s) as a general indicator of the overall quality of the resource;
- 3) reusability (Z_s) of the resource (for example, will a specific resource tool still remain useful if the module where it is used is scaled up by a large factor).

Finally, the average level of knowledge output K is assumed to depend on three factors:

- 1) frequency (X_k) of the use of upstream information for downstream development work;
- 2) frequency (Y_k) of release of new upstream information;
- 3) frequency of instances of rapid downstream assimilation and attendant rework (Z_k) due to modification of upstream information.

Equations (13a)–(13c) are of the Cobb–Douglas type [47], [50], [51] commonly employed in econometric implementations of production functions. H_{int} , S_{int} , and K_{int} are the intercept terms in the regression, and the indices α , β , and γ are dimensionless elasticity parameters of the predictor variables in the model. If each input is scaled by a factor μ , the resulting output scales by μ^r , where $r = \alpha + \beta + \gamma$. Here, r is the *returns to scale* parameter. In this study, we utilize only constant returns to scale, whereby the output is homogeneous of degree 1 (linear homogeneity) in the input quantities, so that $\alpha + \beta + \gamma = 1$. This is a useful assumption, because it ensures that the average and marginal input in each X , Y , and Z are invariant under a change of scale of the output, and therefore, unaffected by any changes in the composition of final H , S , and K . Note that we could also have used a direct linear relationship between the dependent and the independent variables, such as $H = \alpha \bar{X}_h + \beta \bar{Y}_h + \gamma \bar{Z}_h$, in the regression. This type, however, does not exhibit constant or diminishing returns, and is therefore not very useful in practice. Although the linear form may sometimes yield a high value for the coefficient of multiple determination (R^2), it does not necessarily imply that the underlying regression model is reasonable.

In order to estimate the parameters of the model, a multiple regression [52] on (13a)–(13c) is suitable in the form

$$\ln Y = \ln Y_{\text{int}} + \alpha \ln X_1 + \beta \ln X_2 + \gamma \ln X_3 + \epsilon w \quad (14)$$

where Y is the dependent variable, Y_{int} is the intercept, X_i 's are the predictor variables, ϵ is a stochastic disturbance term, and w is a scalar weight that can be adjusted to the level of correlation between the dependent and the predictor variables.

⁵In a specific problem, one may include as many variables as deemed necessary, but the procedure outlined here remains essentially the same.

TABLE II
SAMPLE VALUES OF HUMAN, TECHNOLOGY, AND KNOWLEDGE CONTRIBUTION AMOUNTS FOR C_1

Human				Technology				Knowledge			
Id	X_h	Y_h	Z_h	Id	X_s	Y_s	Z_s	Id	X_k	Y_k	Z_k
1	3	5	4	1	6	6	3	1	4	1	1
2	4	6	4	2	6	4	5	2	3	1	1
3	4	6	2	3	6	4	4	3	2	2	2
4	6	5	6	4	5	4	5	4	1	3	2
5	5	6	3					5	3	1	1

In multiple regression of this type, the possibility of multicollinearity exists, which could inflate the standard errors of the parameter estimates, thus causing the confidence intervals for the true values to be wide. Statistically, we have employed a forward variable selection process and stepwise regression to avoid multicollinearity as far as practicable. Note that, in our simulation, we are interested only in those particular values of the parameters that produce the required goodness of fit of the regression model to the project data points. Although multicollinearity makes it difficult for us to assess partial effects of the input quantities, it does not, however, diminish the goodness of the fit used in predictions.

To implement (13a)–(13c) empirically, we use a stochastic framework in which an additive disturbance term is appended to these equations expressed in the form (14). For a small sample estimation procedure, we cannot assume normality of the distribution of the dependent variable and the error term in (14). In the model, we, therefore, take ϵ to be independently and identically χ^2 distributed with zero expectation value and unit standard deviation. One possible source of disturbance lies in the observation that, in software development projects, optimal assessment of the input variables are generally performed with occasional errors that are distributed randomly in the data. Also, managerial skills and experiences in correctly evaluating and assessing development output vary randomly across the projects of C_X ; the specific causes of these variations are not known to the present author examining the data as an outside evaluator. They affect the intercept parameters, and if these are randomly distributed, then disturbances should include managerial differentials of project estimates [53]. In effect, all the randomization can be lumped into ϵ in (14) representing stochastic fluctuations in the data.

Numerical values of the response variables are represented in an ordinal scale of 1–8, with 1 the smallest and 8 the largest. Some sample values of contributory variables are displayed in Table II for C_1 . “Id” is an identifier set to arbitrary values on a nominal scale to locate an item in the data. It, therefore, identifies a project engineer in the human contribution data, a technology resource in the technology contribution data, and an information instance (for example, a unit of upstream information released downstream or a feedback unit of information propagated from downstream development activities) in the knowledge contribution data. The numerical values of the H estimates are evaluated on an ordinal scale of 1–8 with 1 the lowest and 8 the highest

level. The S estimates are recorded on the same numerical scale. The raw data for the K values are first averaged over 180 development hours (about 25 days of development activity), expanded out to an ordinal scale of 1–8 and then recomputed.

The parameters in the multiple regression model (14) are estimated by bootstrapping the ϵ terms in Monte Carlo trials [54]–[56]. The basic steps are as follows.

- 1) Using the observational data from the projects, obtain least square estimates of the parameters.
- 2) With these estimates, compute residuals as the differences between observed and predicted responses.
- 3) Resample the residuals by drawing random numbers from a uniform distribution by selecting an equal number of points at a time with replacement.
- 4) Compute bootstrapped responses by adding the resampled residuals to the predicted response values.
- 5) Use the bootstrapped versions as surrogates for their values based on the true parameters and regress them on the independent variables to obtain bootstrapped estimates of the parameters.
- 6) Repeat steps 2)–5) a sufficient number of times to generate sampling distributions of the parameters.

In the trials, we keep the R^2 values close to those calculated from the observed data points by adjusting the weight factor in the regression equation. For the human contribution equation, $\alpha = 0.4(0.1)$ and $\beta = 0.3(0.1)$ with $R^2 \approx 0.37$; for the technology contribution, $\alpha = 0.4(0.2)$ and $\beta = 0.3(0.1)$ with $R^2 \approx 0.35$; and for the knowledge contribution, $\alpha = 0.4(0.1)$ and $\beta = 0.2(0.1)$ with $R^2 \approx 0.32$.

The next task is to fix the period of integration of overlapping activities. It is important to make a good choice here, since the accuracy of the numerical work depends upon it. The period should be small enough for accurate convergence to stability, but, at the same time, not so small as to greatly slow down the approach to convergence. By trial and error, we find that a period equal to 40 h of development works well. If a COD project is actually followed in real time through a simulation, the integration period may occasionally need to be adjusted to ensure a good prediction of project output in time. Such an endeavor should be undertaken if, for example, the project is simulated for a total development horizon that spans more than one year.

The final task is to obtain numerical estimates for the overlap parameters in the project matrix. The procedure used here is as follows: first, we determine the total number of *atomic units* of downstream iterations in one component caused by information released upstream in an interdependent component over a single development cycle for integration. An atomic unit of activity is any indivisible or irreducible piece of intellectual accomplishment having a tangible worth in software development. Two real examples from P_X are the following: a developer meeting (lasting 1 h) concerning downstream requirements modification of a small unit of C_2 ; a joint assessment by business analysts and development managers (lasting approximately 2 h) of the minimum number of design changes in a unit of C_3 caused by upstream modifications in the public interfaces of Java classes developed in C_1 . Next, we divide the number of atomic units

TABLE III
SAMPLE VALUES OF OVERLAP INTEGRATION PARAMETERS

α_{11}	0.74	α_{22}	0.81	α_{33}	0.86	α_{44}	0.91	α_{55}	0.81
α_{12}	0.71	α_{23}	0.00	α_{34}	0.71	α_{45}	0.55		
α_{13}	0.89	α_{24}	0.00	α_{35}	0.77				
α_{14}	0.91	α_{25}	0.11						
α_{15}	0.87								

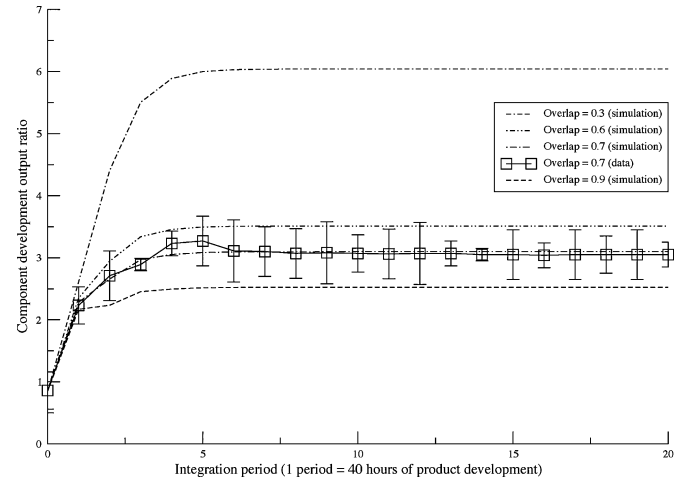


Fig. 3. Development output ratios of C_1 and C_2 : simulations and data.

actually completed during the specified integration period by the managerial estimates of the total number of accomplishable units of tasks for that period. From one period to another this value fluctuates, although not widely. To smooth out these fluctuations, the values are averaged over four consecutive cycles of integration. Some typical values of the parameters computed directly from the project data appear in Table III. As pointed out earlier, self-integration activities, under ideal circumstances, should be unity. Our observational values were in the range 0.70–0.96. Further, the database component exhibited a below-threshold overlap with the GUI and the middle-tier components right in the initial setup phase. Consequently, we keep α_{23} , α_{24} and their symmetric transposed elements zero throughout the simulations.

The entire COD simulation of P_X is designed as a complete, multistage Monte Carlo experiment in which the parameter estimation procedures described earlier are subprocesses, and the observed values of the overlaps are resampled to generate surrogate pseudosamples in the different trials of the experiment by drawing random numbers with replacement from a uniform distribution, so that a random fraction of the actual points are replaced by duplicated values. The final results are computed as average values of the various quantities of interest in the simulation that are realized in the different trials of the Monte Carlo experiment.

3) *Time Evolution and Model Validation:* In Fig. 3, we display the time-domain behavior over 20 integration periods of the development output ratios of C_1 and C_2 for fixed values of the corresponding overlap parameters. Five different overlaps

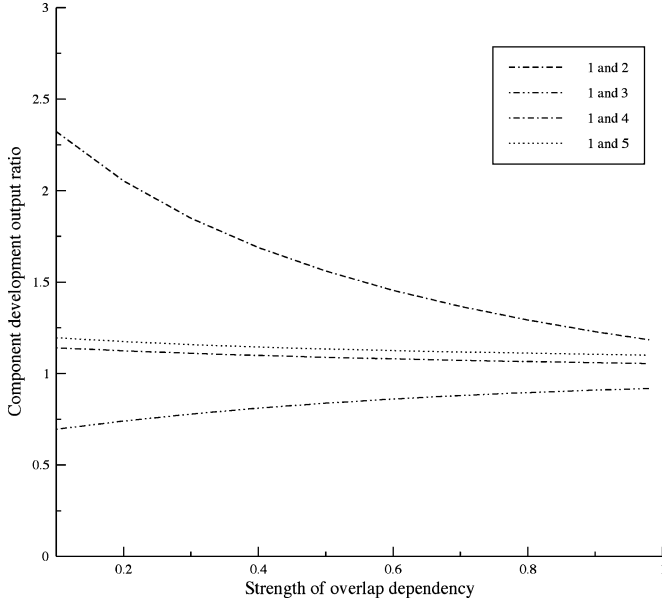


Fig. 4. Overlap dependency simulations of development output ratios of components. Results are shown relative to C_1 only.

are used in the simulations. The model is validated against actual data from the projects corresponding to $\alpha_{12} = 0.71$, which also appears in the graph. The error bars on the actual data points represent statistical fluctuations inherent in the sampling process. Nevertheless, the simulated results lie well within the range of the data. In all the different categories of overlap, the flow to asymptotic convergence in time is steady, and the convergence itself is stable. This is intuitively expected, because in the absence of any external (possibly nonlinear) effects, the overlapping development of C_1 and C_2 (and of the other pairs as well) is unperturbed. Conversely, while the project is in progress, if this ratio can be ascertained (from graphs such as Fig. 3) for specific ranges of integration, then it can be used to manage and distribute resources in the components. Later, we present a more sophisticated method to further demonstrate this behavior.

4) *Component Output Ratios and Model Validation:* Fig. 4 shows the simulation results for the development output ratios of various components corresponding to different values of the overlap. The figure displays the behavior of the output ratios for C_2 , C_3 , C_4 , and C_5 in relation to C_1 , for which the overlap parameters are all nonzero. Each run of the simulation uses fixed values of the various starting amounts. The graph indicates that the development output ratios p_1/p_4 and p_1/p_5 are almost identical and remain sufficiently close to 1 over the entire range of integration. This suggests that these components would be represented with comparable significance in $\mathcal{G}_X$, with similar ranges of dependencies on the upstream released information. Thus, to ensure the stability of P_X , when these two components are considered separately, their resources should be distributed very similarly. This behavior is also observed in the p_1/p_2 and p_1/p_3 ratios for somewhat larger values of the overlap ($\alpha > 0.75$). For relatively smaller overlap, the behavior is different, however. For example, in the range $0.1 < \alpha < 0.6$, since C_1 's output is higher than that of C_2 , the likelihood of

the availability of upstream information released early by C_1 is also high, but because of the low overlap C_2 can proceed without much use of the upstream information released by C_1 . This would not be the case if the overlap were large, and the heavy dependency on the upstream information would greatly alter downstream development activities. The ratios of output, in this case, would tend asymptotically to 1. The graph clearly validates this observation. Similar trend is also evident in the p_1/p_3 ratio, although here the actual variations from small to large overlap are of comparatively short range.

5) *Two-Mode Projections, Phase Diagrams, and Their Use in Project Management:* After validating the model against observed data from the actual projects and further tuning it for accuracy of predictions of future time development, we now provide a utility that greatly benefits project managers to form judgmental estimates of the relative distribution of the resources used in the various components of P_X . It is a common feature of many software development projects that their technology resources, in particular, do not change appreciably relative to the magnitudes of the human and knowledge units, at least over a considerable fraction of the time span of development. It, therefore, seems practicable to halt the growth or decay of this resource and examine the relative changes of the other two resources. This is called a *two-mode projection* analysis of the resource manifold. For projects involving many different resources, smaller, low-dimensional projections elucidate the close relationships of a selected, exclusive group of resources used in the projects. For an arbitrary integration period t_s , assume, therefore, $S_i(t_0) = S_i(t_s)$. Using (2), we obtain

$$P_i(t_s) = H_i^2(t_s) + K_i^2(t_s) = p_i(t_s) - S_i^2(t_0) \quad (15)$$

where i identifies the component C_i of P_X . For two components i and j , the ratio

$$M_{ij}(t_s) = \frac{P_i(t_s)}{P_j(t_s)} = \frac{H_i^2(t_s) + K_i^2(t_s)}{H_j^2(t_s) + K_j^2(t_s)} \quad (16)$$

remains constant for a specific time of integration at which actual data are acquired from the components. In general, however, it evolves as P_X unfolds in time. From (16), we have

$$H_i^2(t_s) - M_{ij}(t_s) H_j^2(t_s) + K_i^2(t_s) - M_{ij}(t_s) K_j^2(t_s) = 0. \quad (17)$$

Our simulation, in this case, is based on this important result. Consider two cases: 1) $H_i^2(t_s) - M_{ij}(t_s) H_j^2(t_s) = m > 0$ and 2) $H_i^2(t_s) - M_{ij}(t_s) H_j^2(t_s) = -m < 0$, where m is a positive number. It is easy to see that, if $H_i(t_s)/H_j(t_s) > \rho$, then $K_j(t_s)/K_i(t_s) > \rho$, where ρ is another positive number. Similarly, if $H_j(t_s)/H_i(t_s) > \rho$, then $K_i(t_s)/K_j(t_s) > \rho$. With the help of these inequalities, we can estimate in the simulations how the early release of upstream information may be generated in proportion to the amounts of human resources in downstream development activities. This knowledge is extremely useful in the management of allocation, distribution, and use of the resources in the components.

Now, for different values of the integration time, ρ yields a “phase transition” trajectory, which demarcates the boundary of the regions where the ratios of the human and the knowledge

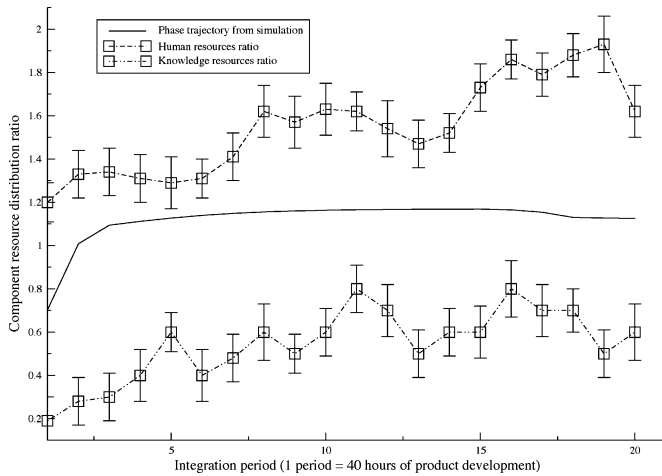


Fig. 5. Actual resource-ratio distributions and the phase trajectory from simulation.

resources in a two-mode projection are significant. The possibility of a phase shift introduces a *scale* in the dynamics, characterized by ρ . Depending on the value of this scale parameter for a specific evolution in real time, a slight flip in the ratio H_i/H_j or K_i/K_j could cause a transition of the region of significance from one side of the phase line to the other. Thus, if the ratio H_i/H_j is maintained for stable evolution so as to lie on one specific side of the phase line, the corresponding ratio K_i/K_j must be restricted to lie on the opposite side of the phase line. If this condition is not satisfied, then the state of the project will be unstable. For example, if the proportion of human resources in two components coexisting in a two-mode projection is more significant than that of the released knowledge resources in these projects, then the project low in human resources may proceed stably by releasing upstream information early to the one with enhanced human efficiency. Due to the presence of the phase line, this always acts unidirectionally. In other words, if both H_i/H_j and K_i/K_j lie on the same side of the phase line, the state of the COD project will be unstable in that integration period. In order to ensure stable coexistence, the resources must be distributed in such a way that their relative proportions belong on opposite sides of the phase line. However, at some later period, if the significance starts to shift, the same phase line will again dictate which project may release early upstream information so as to facilitate downstream activities in the other.

Fig. 5 illustrates the earlier effects in the simulation. Here, we show a two-mode projection of the human and knowledge resources in C_1 and C_2 , together with associated data from the actual projects. The overlap parameter is controlled at 0.71 throughout the simulation at which the model has been previously validated. The data for the resource contributions are produced at each designated integration period (20 periods are shown) using (13a) and (13c). Although the errors are somewhat large, throughout most of the integration time shown in the graph, the human and knowledge resources coexist on opposite sides of the phase line ensuring the stability of P_X , at least for about the first six months of development time. The simulation can be continued for longer time as the project pro-

gresses (the linear model will of course be inaccurate after a while), and by observing the trend in the evolution of the resources, the phase diagram can again be employed to manage and redistribute them, if necessary, to guarantee a continued stability of P_X . Should the need arise to use additional resources in the project, similar two-mode (and, for more extensive computations, even three-mode) projections can be constructed.

VI. CONCLUSION AND FUTURE DIRECTIONS

In this section, we point out some limitations of the present model and indicate a few directions of further research in this field.

The program of this work is simple. Using a *deterministic* evolution equation, we characterize the asymptotically stable, temporal behavior of the COD project P_X , conceptualized as a vector in a multidimensional project space determined by its components. Although the evolution dynamics is fully deterministic in character, the equation contains a number of parameters that are only resolved *stochastically*. By means of multiple regression procedures, estimates of the parameters are obtained from observational data collected from actual industrial projects. The model is validated and tuned with these estimates, and then it is used to compute bimodal projections of resource distributions and phase trajectories in project space that reconcile the stability of evolution with the resource distribution patterns for arbitrary times. The model has predictive power within its domain of validity, and the knowledge acquired from its analysis is very useful in making critical decisions in project management involving timely allocation and deallocation of resources in a COD project.

There is yet another mathematical framework that uses an intrinsically stochastic approach. The deterministic evolution equations used in the present model can be easily converted into stochastic equations by adding random disturbance terms, which may be modeled with white noise or any suitable function with a well-defined probability distribution. The various quantities of interest can then be computed as expectation values obtained by smeared integrals over the appropriate probability distribution function. This method involving stochastic differential equations, albeit more involved in computations, is straightforward to apply in principle. The time span of validity of the dynamics used in the present model is typically six months. In this short time, various uncertainties in P_X are better modeled by stochastically resolved parameters than by the use of a general probability distribution, since the project uncertainties at this stage are mostly task-specific, affecting the component overlap activities only locally. The intrinsically stochastic approach yields better results for development periods in which the local uncertainties have either died out or at least stabilized to a great extent. Any residual random variations in the evolution can then be captured by a general stochastic source affecting the global evolution of project output at the level of expectation values.

The fact that the present model fits actual data reasonably well does not, however, justify its overall validity nor protect it from deviations. The model is indeed a good description for a specific period of time within the restricted domain of tuned

parameters. Even so, it is not to be extrapolated injudiciously. The comparison of the model to actual numerical data does, in fact, identify the limitations and flaws in the model. It breaks down at longer periods of time, primarily because the underlying assumptions in the model become largely invalid in this period due to a number of new effects. Incorporating these effects, and subsequently, modifying the present assumptions is vital to the use of the model for long-term predictions. This is neither a matter of statistical testing nor of historical data fit but is indubitably a value judgement that the project modeler and the concerned managers must together make to render the resultant model suitable for policy design and decision-making. It is to address some of these concerns that we now turn.

There are a number of simplifying assumptions in the project dynamics and the accompanying simulations. In a more generalized approach, some of these assumptions can be forgone. The basic, first step we have taken in this study to formalize the COD dynamics of P_X is to construct an analytical model in terms of which it is characterized by a linearly unfolding dynamical system. We have shown that, at least for a relatively short development span, with the project progressing through periods of integration with synchronization points at each period, nonlinear effects may be ignored. Marked deviations from linearity manifest themselves in the simulations beyond our limited domain of study.

We have also mentioned that the present theoretical model works under the assumption of a symmetric project matrix. Although this was a simplification in the *implementation* of the model, the actual project data were not too deviant and supported this assumption to some extent. However, this assumption is not binding in any respect and can be easily relaxed. To estimate the overlap parameters embodied in the project matrix in a general unconstrained system, the statistical likelihood ratio (LR) procedure can be applied. This amounts to estimating the unconstrained model ($\alpha_{ij} \neq \alpha_{ji}$) by the maximum likelihood (ML) technique and then estimating the constrained model ($\alpha_{ij} = \alpha_{ji}$) also by the same ML technique. If $\ln L_c$ and $\ln L_u$ are the maximized log-likelihood functions for the constrained and unconstrained models, respectively, then the LR test statistic $-2 \ln L_c / L_u$ is distributed asymptotically as a χ^2 random variable. This estimation procedure can be easily implemented in software and tested for validity of the model predictions against observed data from COD projects.

One common nonlinear effect that may be directly incorporated into the present dynamics involves misuse, loss, and decay of productive development output. This usually happens for various reasons, including employees leaving the project, problems of integration with third-party, commercial software products used in the project, incorrect, inappropriate, or inefficient technological resources used, lack of technical knowledge in certain areas of development, unavailability of proper documentation of released information in upstream development, etc. Each component, however, is affected with a different strength by this effect.

In this analysis, the overlap dependencies are all taken to be time independent. In the actual projects that motivated the COD model of P_X , it was observed that the overlap amounts were determined within tolerable limits of uncertainty immediately

after $\mathcal{G}_X$ was put into perspective. Thereafter, these remained largely unchanged. We further noted that the fine-tuning necessary for correct integration was only a local activity that did not introduce major time variations in the parameters, since, for the first six months of product development, the *magnitudes* of the intercomponent dependencies were not observed to vary substantially with time as the project unfolded through continuous downstream iterations. This practice could have been adopted as a major convenience for project management at least for the initial periods, but it certainly was not a universal phenomenon and could not have been promised for longer time frames. Even in this restrictive scenario, some unavoidable, small variations were indeed observed, which were customarily absorbed within error limits. To overcome this limitation, a major enhancement of the present model would require incorporating time-dependent overlaps in the dynamics. Thus, to confront a more ambitious, more accurate, and longer time frame simulation, a useful technique is to initially obtain a first-order solution very similar to the present one, and then to start adding small time variations in the overlaps and solving the resulting dynamics perturbatively in a time-series expansion around the time-independent solution. Numerical simulations on the computer are eminently suited for this kind of perturbative approach. These important enhancements are presently being investigated by the author.

APPENDIX

To prove that η is approximated by $|\vec{P}(n+1)|/|\vec{P}(n)|$ as given by (12), it is enough to establish the case for the simplest 2-D model. Suppose, then, $\vec{P}$ is a two-component vector $\vec{P}(k) = (p_1(k), p_2(k))$, and denote $\vec{V}_i = (v_{1i}, v_{2i})$. First, using (9), we write

$$\begin{aligned} p_1(k) &= c_1 \lambda_1^k v_{11} + c_2 \lambda_2^k v_{12} \\ p_2(k) &= c_1 \lambda_1^k v_{21} + c_2 \lambda_2^k v_{22}. \end{aligned}$$

Here, λ_1 and λ_2 are the two distinct eigenvalues of the project matrix, and suppose that λ_2 corresponds to the spectral radius. We, therefore, need to show that $[p_1^2(k+1) + p_2^2(k+1)]^{1/2} / [p_1^2(k) + p_2^2(k)]^{1/2}$ approximates λ_2 in the asymptotic limit. Thus,

$$\begin{aligned} & \frac{p_1^2(k+1) + p_2^2(k+1)}{p_1^2(k) + p_2^2(k)} \\ &= \frac{c_1^2 \lambda_1^{2(k+1)} v_{11}^2 + c_2^2 \lambda_2^{2(k+1)} v_{12}^2 + 2c_1 c_2 \lambda_1^{k+1} \lambda_2^{k+1} v_{11} v_{12}}{c_1^2 \lambda_1^{2(k+1)} v_{21}^2 + c_2^2 \lambda_2^{2(k+1)} v_{22}^2 + 2c_1 c_2 \lambda_1^{k+1} \lambda_2^{k+1} v_{21} v_{22}} \\ &= \lambda_2^{2(k+1)} \left[c_1^2 \left(\frac{\lambda_1}{\lambda_2} \right)^{2(k+1)} v_{11}^2 + c_2^2 v_{12}^2 \right. \\ & \quad \left. + 2c_1 c_2 \left(\frac{\lambda_1}{\lambda_2} \right)^{k+1} v_{11} v_{12} \right. \\ & \quad \left. + c_1^2 \left(\frac{\lambda_1}{\lambda_2} \right)^{2(k+1)} v_{21}^2 + c_2^2 v_{22}^2 + 2c_1 c_2 \left(\frac{\lambda_1}{\lambda_2} \right)^{k+1} v_{21} v_{22} \right] \\ & \xrightarrow{k \rightarrow \infty} \lambda_2^{2(k+1)} c_2^2 (v_{12}^2 + v_{22}^2). \end{aligned}$$

Similarly,

$$p_1^2(k) + p_2^2(k) \xrightarrow{k \rightarrow \infty} \lambda_2^2 c_2^2 (v_{12}^2 + v_{22}^2).$$

Taking the ratio, we see that

$$\left[\frac{p_1^2(k+1) + p_2^2(k+1)}{p_1^2(k) + p_2^2(k)} \right]^{\frac{1}{2}} \approx \lambda_2.$$

The general N -dimensional case follows without further complications.

ACKNOWLEDGMENT

The author wishes to acknowledge and thank two anonymous referees and the departmental editor of IEEE TEM for suggesting several improvements in the paper. Thanks are also due to Dr. M. Day for helpful discussions and comments.

REFERENCES

- [1] P. G. Smith and D. G. Reinertsen, *Developing Products in Half the Time. New Rules, New Tools*. New York: Van Nostrand/Reinhold, 1998.
- [2] S. Minderhouda and P. Fraser, "Shifting paradigms of product development in fast and dynamic markets," *Rel. Eng. Syst. Safety*, vol. 88, no. 2, pp. 127–135, 2005.
- [3] H. Takeuchi and I. Nonaka, "The new product development game," in *The Product Development Challenge—Competing through Speed, Quality, and Creativity*, K. B. Clark and S. C. Wheelwright, Eds. Boston, MA: Harvard Bus. School Press, 1995.
- [4] V. Krishnan, S. Eppinger, and D. Whitney, "A model-based framework to develop product development activities," *Manag. Sci.*, vol. 43, no. 4, pp. 437–451, 1997.
- [5] R. M. Eastman, "Engineering information release prior to final design freeze," *IEEE Trans. Eng. Manag.*, vol. EM-27, no. 2, pp. 37–41, May 1980.
- [6] E. Mansfield, J. Rapoport, J. Schnee, S. Wagner, and M. Hamburger, *Research and Innovation in the Modern Corporation*, New York: Norton, 1971.
- [7] V. Krishnan, "Managing the simultaneous execution of coupled tasks in concurrent product development," *IEEE Trans. Eng. Manag.*, vol. 43, no. 2, pp. 210–217, May 1996.
- [8] W. Lincke, *Simultaneous Engineering*. Munich, Germany: Hanser, 1995.
- [9] A. Griffin, "The impact of engineering design tools on new product development efficiency and effectiveness," in *Proc. 3rd EISM Int. Prod. Develop. Conf.*, Fontainebleau, France, 1996, pp. 363–380.
- [10] S. D. Eppinger, "Model-based approaches to managing concurrent engineering," *J. Eng. Des.*, vol. 2, no. 4, pp. 283–290, 1991.
- [11] S. D. Eppinger, D. E. Whitney, R. P. Smith, and D. A. Gebala, "A model-based method for organizing tasks in product development," *Res. Eng. Des.*, vol. 6, pp. 1–13, 1994.
- [12] J. D. Blackburn, "New-product development: The new time wars," in *Time-Based Competition—The Next Battleground in American Manufacturing*, J. D. Blackburn, Ed. Homewood, IL: Business One Irwin, 1991.
- [13] K. Imai, I. Nonaka, and H. Takeuchi, "Managing the new product development process: How the Japanese companies learn unlearn," in *The Uneasy Alliance*, K. B. Clark, R. H. Hayes, and C. Lorenz, Eds. Boston, MA: Harvard Business School Press, 1985.
- [14] A. Ha and E. Porteus, "Optimal timing of reviews in concurrent design for manufacturability," *Manag. Sci.*, vol. 41, no. 9, pp. 1431–1447, 1995.
- [15] J. Galbraith, *Designing Complex Organization*. Reading, MA: Addison-Wesley, 1973.
- [16] M. L. Tushman and D. A. Nadler, "Information processing as an integrating concept in organizational design," *Acad. Manag. Rev.*, vol. 3, no. 3, pp. 613–624, 1978.
- [17] R. Cordero, "Managing for speed to avoid product obsolescence: A survey of techniques," *J. Prod. Innov. Manag.*, vol. 8, no. 4, pp. 289–294, 1991.
- [18] K. M. Eisenhardt and B. N. Tabrizi, "Accelerating adaptive processes: Product innovation in the global computer industry," *Adm. Sci. Quart.*, vol. 40, pp. 84–110, 1995.
- [19] C. H. Loch and C. Terwiesch, "Communication and uncertainty in concurrent engineering," *Manag. Sci.*, vol. 44, no. 8, pp. 1032–1048, 1998.
- [20] C. Terwiesch and C. Loch, "Measuring effectiveness of overlapping development activities," *Manag. Sci.*, vol. 45, no. 4, pp. 455–465, 1999.
- [21] K. Clark and T. Fujimoto, *Product Development Performance, Strategy, Organization, and Management in the World Auto Industry*. Boston, MA: Harvard Business School Press, 1991.
- [22] J. Hu and Y. Peng, "Multidisciplinary knowledge modeling and cooperative design for automobile development," in *Cooperative Design, Visualization, and Engineering. Lecture Notes in Computer Science*, Berlin, Germany: Springer-Verlag, 2007, pp. 304–306.
- [23] J. Sobieszczanski-Sobieski and R. T. Haftka, "Multidisciplinary aerospace design optimization: survey of recent developments," *Struct. Multidiscip. Opt.*, vol. 14, no. 1, pp. 1–23, 1997.
- [24] M. Bandecchi B. Melton F. Ongaro. (1999, Sep.). Concurrent engineering applied to space mission assessment and design. *ESA Bul.* [Online]. Available: <http://esapub.esrin.esa.it/bulletin/bullet99/bande99.pdf>
- [25] J. M. Davis L. K. Keys I. J. Chen. (2004, Feb.). Collaborative engineering for research and development. *NASA Rep.* [Online]. Available: <http://gltrs.grc.nasa.gov/reports/2004/TM-2004-212965.pdf>
- [26] M. R. Cutkosky, J. M. Tenenbaum, and J. Glicksman, "Madefast: Collaborative engineering over the internet," *Commun. ACM*, vol. 39, no. 9, pp. 78–87, 1996.
- [27] W. D. Li, J. Y. H. Fuh, and Y. S. Wong, "An internet-enabled integrated system for co-design and concurrent engineering," *Comput. Ind.*, vol. 55, no. 1, pp. 87–103, 2004.
- [28] B. Eynard, S. Liénard, S. Charles, and A. Odinat, "Web-based collaborative engineering support system: Applications in mechanical design and structural analysis," *Concurr. Eng.*, vol. 13, no. 2, pp. 145–153, 2005.
- [29] Y. E. Nahm and H. Ishikawa, "An internet-based integrated product design environment. Part ii: Its applications to concurrent engineering design," *Int. J. Adv. Manuf. Technol.*, vol. 27, no. 5–6, pp. 431–444, 2006.
- [30] J. D. Blackburn, G. Hoedemaker, and L. N. V. Wassenhove, "Concurrent software engineering: Prospects and pitfalls," *IEEE Trans. Eng. Manag.*, vol. 43, no. 2, pp. 179–188, May 1996.
- [31] M. A. Cusumano and R. W. Selby, *Microsoft Secrets: How the World's Most Powerful Software Company Creates Technology, Shapes Markets, and Manages People*. New York: Free Press, 1995.
- [32] M. A. Cusumano S. A. Smith, "Beyond the waterfall: Software development at Microsoft," in *Competing in the Age of Digital Convergence*, D. B. Yoffie, Ed. Boston, MA: Harvard Business School Press, 1997.
- [33] J. P. Murray, "Reducing it project complexity," *Inform. Strategy Exec. J.*, vol. 16, no. 3, pp. 30–38, 2000.
- [34] C. Chen and L. Lin, "Decomposition of interdependent task group for concurrent engineering," *Comput. Ind. Eng.*, vol. 44, no. 3, pp. 435–459, 2003.
- [35] R. Henderson and K. Clark, "Architectural innovation: The reconfiguration of existing product technologies and the failure of established firms," *Admin. Sci. Quart.*, vol. 35, no. 1, pp. 9–30, 1990.
- [36] V. Krishnan and K. Ulrich, "Product development decisions. a review of the literature," *Manag. Sci.*, vol. 47, no. 1, pp. 1–21, 2001.
- [37] V. L. Mitchell and B. R. Nault, "Cooperative planning, uncertainty, and managerial control in concurrent design," *Manag. Sci.*, vol. 53, no. 3, pp. 375–389, 2007.
- [38] S. Chen, "An integrated methodological framework for project task coordination and team organization in concurrent engineering," *Concurr. Eng.*, vol. 13, no. 3, pp. 185–197, 2005.
- [39] K. Clark and T. Fujimoto, "Overlapping problem-solving in product development," in *Managing International Manufacturing*, K. Ferdows, Ed. Amsterdam, The Netherlands: Elsevier/North-Holland, 1989, pp. 127–152.
- [40] S. C. Wheelwright and K. B. Clark, *Revolutionizing Product Development*. New York: Free Press, 1992.
- [41] D. N. Ford and J. D. Sterman, "Overcoming the 90 management in concurrent development projects," *Concurr. Eng.*, vol. 11, no. 3, pp. 177–186, 2003.
- [42] J. Galbraith, "Organization design: An information processing view," *Interfaces*, vol. 4, no. 3, pp. 28–36, 1974.
- [43] P. C. Matthews, *Vector Calculus*. London, U.K.: Springer-Verlag, 1998.
- [44] T. S. Blyth and E. E. Robertson, *Further Linear Algebra*. London, U.K.: Springer-Verlag, 2002.
- [45] S. H. Strogatz, *Nonlinear Dynamics and Chaos*. Reading, MA: Addison-Wesley, 1994.
- [46] J. D. Sterman, *Business Dynamics: Systems Thinking and Modeling for a Complex World*. Boston, MA: Irwin/McGraw-Hill, 2000.

- [47] E. R. Berndt, *The Practice of Econometrics. Classic and Contemporary*. Reading, MA: Addison-Wesley, 1991.
- [48] J. T. Sandefur, *Discrete Dynamical Systems—Theory and Applications*. Oxford, U.K.: Clarendon, 1990.
- [49] J. M. Ortega, *Matrix Theory*. New York: Plenum, 1987.
- [50] C. Cobb and P. H. Douglas, "A theory of production," *Amer. Econ. Rev.*, vol. 18, no. 1, pp. 139–165, 1928.
- [51] P. A. Samuelson, "Paul Douglas' measurement of production functions and marginal productivities," *J. Polit. Econ.*, vol. 87, no. 5, pp. 923–939, 1979.
- [52] W. D. Berry and S. Feldman, *Multiple Regression in Practice*. Beverly Hills, CA: Sage, 1985.
- [53] M. B. McElroy, "Additive general error models for production, cost, and derived demands or share equations," *J. Pol. Econ.*, vol. 95, no. 4, pp. 737–757, 1987.
- [54] C. Z. Mooney, *Monte Carlo Simulation*. Thousand Oaks, CA: Sage, 1997.
- [55] C. Z. Mooney and R. D. Duval, *Bootstrapping: A Nonparametric Approach to Statistical Inference*. Newbury Park, CA: Sage, 1993.
- [56] W. H. Press, B. P. Flannery, S. A. Teukolsky, and W. T. Vetterling, *Numerical Recipes in C: The Art of Scientific Computing*. Cambridge, U.K.: Cambridge Univ. Press, 1992.



Jaideep Ghosh received the B.Sc. degree in physics (with honors), mathematics, and statistics and the M.Sc. degree in physics from the University of Calcutta, Kolkata, India, and the Ph.D. degree in theoretical physics and the M.S. degree in information science from the University of Pittsburgh, Pittsburgh, PA.

He was in the industry for several years and was involved in software research and development, and also in telecommunications. He was with the Computer Information Systems and Technology Department, University of Maryland University College, Adelphi, MD, where he was engaged in teaching. He is currently a Faculty at the Information Systems and Operations Management Department, School of Management, George Mason University, Fairfax, VA. His current research interests include dynamic systems modeling, simulations and statistical analysis of business applications, with focus on distributed management, offshore outsourcing, and organizational intelligence and knowledge management.